example, an application may have a reactive number that needs to be updated in response to every click or some other action by the user. If the placeholder for this number is nested deep under many tags, with SolidJS there is no need to worry about the complete re-rendering of the entire component. The framework has the ability to update only the reactive value without affecting any other tags around it. This type of granular control makes the application react to the user action instantly.

- **Easy to use**: The learning curve associated with it is also a decisive factor in choosing a particular framework. SolidJS has been designed in such a manner that it is easy to use even for a developer with no previous experience in other frameworks. This low experience adaptability ensures ease of use.
- **Well-rounded:** SolidJS is a well-rounded framework. It offers various important features such as caching, reactivity, state management, and data fetching. And it is designed in such a way that it can be the one stop web framework for all your application needs.

Due to these advantages, SolidJS offers the ability to build applications quickly and in an effective way.

Solid primitives

The basic building blocks of applications are called 'solid primitives'. It is essential to have some knowledge about these primitives in order to build responsive applications.

The solid primitives are listed below:

- createSignal
- createEffect
- createMemo
- createResource
- createRoot
- createRenderEffect
- createDeferred
- createComputed
- createContext
- createMutable

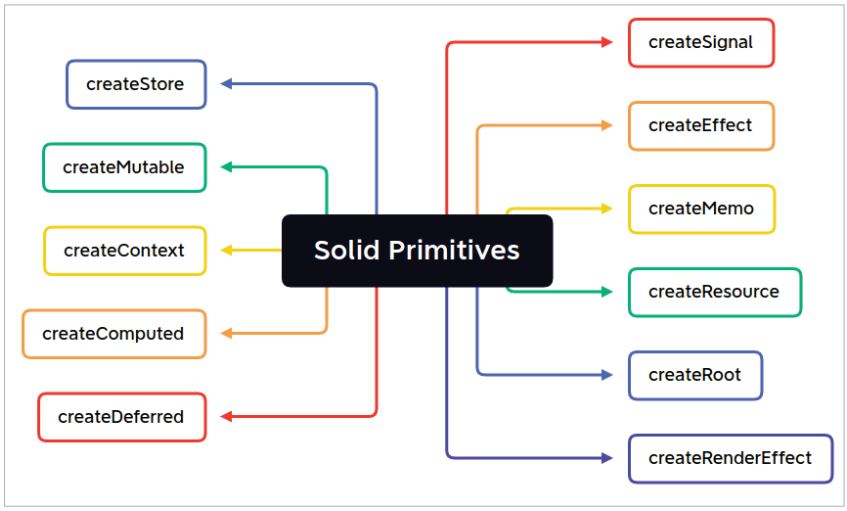

Figure 2: Solid primitives

- createStore

Let's explore a few of these.

createSignal

The most fundamental primitive used with SolidJS is createSignal. The purpose is to build a reactive state variable. An initial value is optional here. It returns a tuple, which consists of a *getter* function to receive the value and a *setter* function to set the value. createSignal can be used with any data type. A simple example is as follows:

```
import { createSignal } from "solid-js";
const [count, setCount] =
createSignal(0);
function increment() {
  setCount(count() + 1);
}
function decrement() {
  setCount(count() - 1);
}
function Counter() {
  return (
    <div>
      <button onClick={decrement}>-</
button>
      <span>{count()}</span>
      <button onClick={increment}>+</
button>
    </div>
  );
}
```

This example has a state variable named *count*. The two functions *increment* and *decrement* are used to update the state variable. The component *counter* is used to display the current value.

createResource

The createResource Solid primitive is used to build a signal that will be tied with an asynchronous function. An interesting example using createResource is as follows:

```
import { createSignal, createEffect }
from "solid-js";
const [count, setCount] =
createSignal(0);
createEffect(() => {
  console.log("Count has been updated
: "count());
});
function Counter() {
  return (
    <div>
      <button onClick={() =>
setCount(count() - 1)}>-</button>
      <span>{count()}</span>
      <button onClick={() =>
setCount(count() + 1)}>+</button>
    </div>
  );
}
```